

Phase 0 Master Foundation Specification

Detailed implementation instructions for Steps 1-4: folder structure, route structure, tenancy rules, and terminology cleanup

Purpose

Phase 0 creates the structural rules the entire rebuild must follow. Nothing in this document is a cosmetic preference. These decisions prevent duplicated modules, leaked tenant data, mismatched URLs, incompatible APIs, and legacy terminology from entering the new codebase.

Document Control	Value
Status	Master specification - implementation ready after final owner decisions
Scope	Phase 0 only: Steps 1-4 of the ten Phase 0 foundation tasks
Applies To	Frontend, backend, database models, routes, APIs, tests, navigation, portals, and future add-ons
Primary Source	Current app architecture audit, route map, module dependency map, and agreed rebuild terminology
Out of Scope	Building business features such as Orders, Webstores, Wrap Lab, payroll, reports, or AI tools

How to Use This Master Document

This document is the Phase 0 build contract. The builder should complete the steps in order, keep a written implementation log, and stop at each approval gate before moving to the next step. No new business module should be rebuilt until all Phase 0 exit criteria are satisfied.

Step	Master Deliverable	Why It Comes First
1. Final Folder Structure	Module-oriented application structure and import rules	Prevents logic from being scattered by page type or feature accident.
2. Final Route Structure	Canonical routes, access zones, redirects, and guard rules	Prevents duplicate navigation, broken deep links, and ambiguous page ownership.
3. Tenant / Organization Ownership Rules	One tenant-scoping standard for data, APIs, storage, and settings	Prevents data leaks and inconsistent ownership across modules.
4. Terminology Cleanup	Canonical vocabulary and legacy migration plan	Prevents the rebuild from reintroducing “Job Ticket” and other conflicting language.

Phase 0 Non-Negotiables

- Do not move files blindly. Every moved file must have a clear owner module, a verified import path, and a replacement or removal decision.
- Do not introduce new top-level routes without adding them to the approved route map in this document.
- All tenant-scoped reads and writes must be enforced in the backend. Hiding data in the frontend is not tenant isolation.
- New code must use canonical terminology even while legacy aliases temporarily remain for migration compatibility.
- No module may create its own duplicate settings, notification, attachment, role, activity-log, or customer model without an explicit shared-system decision.
- No feature work is complete until unit/integration checks and manual route checks are recorded.

Phase 0 Exit Gate

- The codebase has one approved module-oriented folder structure.
- All active routes are classified as internal, portal, public, authentication, or platform-admin routes.
- Every tenant-scoped entity has a defined ownership model and backend tenant filter.
- Legacy terms are mapped, deprecated, and blocked from new feature code.
- Old files/routes marked for deletion are not silently retained as unexplained duplicates.
- The builder has produced a short completion report with file moves, route changes, migration notes, test results, and unresolved decisions.

Step 1 - Final Folder Structure

Objective: create one final, module-oriented repository structure that makes ownership obvious. The current architecture audit shows the codebase has drifted toward page-type groupings and scattered feature files. The rebuild must organize code by module first, then by technical role within that module.

Required Decision

The new application uses a module-first structure. A route page is allowed to be thin composition only. Module business logic, module-only components, API handlers, hooks, and schemas must live with the module that owns them.

1.1 Folder Structure Principles

- A module owns its own pages, components, hooks, services, types, validation, tests, and module-specific documentation.
- Shared code is reserved for genuinely shared primitives used by three or more modules. Do not promote code to shared merely because it might be reused someday.
- Route files compose the correct module page and enforce route-level guard behavior. Route files should not contain the actual business workflow.
- Backend routes, services, models, and background work use the same module ownership boundaries as the frontend.
- Legacy code lives in a clearly labeled temporary area only while a migration is active. It cannot remain as a permanent second implementation.
- Portals are access zones, not duplicate data systems. Portal UI belongs under portal-specific route/layout areas; portal business logic still uses the source module services.

1.2 Required Target Structure

```

src/
  app/
    (auth)/          # Route composition, layouts, access-zone shells only
    (internal)/      # Login, reset-password, account-suspended
    (customer-portal)/ # Tenant staff application shell
    (employee-portal)/ # Customer-facing portal shell
    (webstore-owner)/ # Employee-facing portal shell
    (platform-admin)/ # Owner portal and onboarding shell
    (public)/        # Platform admin shell
    (marketing)/      # Marketing, public forms, public storefront
  modules/           # All business modules, module-owned code
    customers/
    quotes/
    orders/
    production/
    inventory-purchasing/
    pricing/
    finance-billing/
    team-payroll/
    productivity/
    doculink/
    webstores/
    wrap-lab/
    ai-tools/
    community/
    reporting-analytics/
    onboarding/
    platform-admin/
  shared/             # Cross-module primitives only
    components/
    layouts/
    lib/
    hooks/
    types/
    constants/
    validators/
    ui/
  server/
    modules/          # Server code grouped by the same module names
    shared/           # auth, tenant scope, storage, event bus, integrations
  legacy/             # Temporary migration-only quarantine, no new code
  tests/              # Cross-module and end-to-end tests
  docs/               # Specs, decisions, migrations, module documentation

```

1.3 Standard Module Internal Layout

```

modules/orders/
  pages/             # Order list/detail/create pages used by route composition
  components/        # Components used only by Orders
  features/          # Workflow-specific units, e.g., create-order, order-status
  hooks/             # Module-only hooks
  api/              # Client API wrappers for the module
  services/          # Business orchestration, client-side only where applicable
  types/            # Module types and DTOs
  validation/        # Schemas and validation rules
  utils/            # Module-specific calculations/helpers
  tests/            # Unit/integration tests for this module
  README.md         # Module map and ownership notes

```

1.4 Ownership Rules: Module vs Shared

Put It Here	When This Is True	Examples
modules/[module]	Used primarily by one bounded module or requires module business context.	Order status transition rules, wrap inspection form, webstore payout workflow.
shared/components	A UI primitive is used by 3+ modules and contains no module business rules.	Modal, date picker, data table shell, file picker shell, confirmation dialog.
shared/lib or shared/services	A cross-cutting technical capability has one clear universal behavior.	HTTP client, date helpers, formatting, permissions helpers, event emitter adapter.
server/shared	A backend concern applies across modules and has no single feature owner.	Tenant scope middleware, authentication middleware, storage adapter, email transport.
legacy/	A file must exist temporarily for migration/redirect compatibility and has a written retirement task.	Legacy JobTicket redirect adapter, deprecated PricingPage alias.

1.5 Required Module Boundaries

Module	Owns	Must Reuse Instead of Rebuilding
Customers	Customer record, contact data, customer relationship history.	Messaging, documents, forms, portal access, activity logs.
Orders	Order shell, status, Order Items, conversion behavior.	Customer, pricing engine, documents, notifications, production task creation.
DocuLink	Document metadata, templates, questionnaire records, access rules.	File storage, messaging delivery, customer/order relationship IDs.
Webstores	Store setup, store configuration, per-store catalog, storefront workflow.	Customers, orders, payments, documents, notifications, shared product templates.
Wrap Lab	Vehicle-wrap-specific Order Item workflow and records.	Orders, Order Items, files, documents, proofs, production, schedule, customer portal.
AI Tools	Prompt execution, AI result records, credit consumption request flow.	User permissions, usage logs, file storage, activity log, messaging.

1.6 Implementation Sequence

1. Inventory the current repository and classify every major folder/file as: keep and move, rebuild later, temporary compatibility, or delete.
2. Create the target folders without moving feature logic yet.
3. Create module README files that state ownership, dependencies, active routes, and migration status.
4. Move shared primitives first only when they are actually cross-module and tested.
5. Move one module at a time. After each move, run imports, route checks, and relevant tests.
6. Move deprecated files to legacy/ only when a compatibility adapter or redirect still needs them.
7. Delete dead duplicate files after verifying no import, route, API, or test references remain.

1.7 Folder Structure Completion Checklist

- Every active business feature has one module owner.
- No module page contains an unexplained copy of another module's logic.
- No duplicate context/provider folders remain for the same concern.
- No generic "pages" bucket is used as a long-term home for module business logic.
- No new code imports from legacy/ except named compatibility adapters.
- Imports use approved aliases and do not cross modules through relative-path mazes.
- Every module has a README with owner, purpose, inbound dependencies, outbound dependencies, and migration state.

Step 2 - Final Route Structure

Objective: define one canonical route map so each page has a clear access zone, URL, guard, breadcrumb position, and module owner. Routes are contracts. They must not be invented ad hoc by individual features.

Required Decision

Use route namespaces by access zone. Internal tenant routes, external portal routes, public storefront routes, and platform-admin routes must never overlap merely because they render similarly.

2.1 Route Design Rules

- Use lowercase kebab-case URL segments. Example: `/customer-portal/orders/:orderId`.
- Use stable nouns for resources and verbs only for explicit action pages where a modal or command cannot serve the need.
- An internal route must live under the internal app shell and require authenticated tenant membership.
- Every portal has a separate top-level URL namespace and its own layout/nav. A portal must never reuse internal navigation.
- Route pages are thin. They load the correct module page, run the approved guard, and pass route parameters.
- No route may represent two unrelated concepts. Example: `/orders` must not secretly contain inventory and purchasing as peer modules.
- All legacy routes must be either redirected to a canonical route or removed. No undocumented duplicate route survives.

2.2 Canonical Access Zones

Access Zone	Route Prefix	Who Uses It	Examples
Authentication	<code>/login</code> , <code>/reset-password</code> , <code>/account-suspended</code>	Unauthenticated or recovering users	Login, password reset, account recovery.
Internal Tenant App	<code>/dashboard</code> and direct internal routes	Shop owner, admin, staff with tenant membership	Orders, customers, pricing, production, billing.
Customer Portal	<code>/customer-portal/*</code>	Customer users only	Orders, proofs, invoices, payments, documents, forms.
Employee Portal	<code>/employee-portal/*</code>	Employee users only	Timeclock, schedule, tasks, assigned work, pay.
Webstore Owner	<code>/webstore-owner/*</code> and <code>/owner-portal/*</code>	Webstore owners and managers	Onboarding, store dashboard, approval, payout visibility.
Public	<code>/store/:storeSlug</code> and approved public pages	Anyone with public access	Storefront, public questionnaire, marketing pages.
Platform Admin	<code>/platform-admin/*</code>	Platform administrators only	Tenant management, analytics, email logs, platform settings.

2.3 Canonical Internal Route Map

Module	Canonical Routes	Notes
Home	<code>/dashboard</code>	Main tenant dashboard only. Do not create duplicate dashboard routes for every top-level nav section unless they are module dashboards.
Customers	<code>/customers</code> , <code>/customers/new</code> , <code>/customers/:customerId</code>	Customer detail uses a real route, not a modal-only page.
Quotes	<code>/quotes</code> , <code>/quotes/new</code> , <code>/quotes/:quoteId</code>	Quote converts into Order from the detail page.
Orders	<code>/orders</code> , <code>/orders/new</code> , <code>/orders/:orderId</code>	Order Item detail is nested under its parent Order.
Order Items	<code>/orders/:orderId/items/:orderItemId</code>	Canonical replacement for legacy Job Ticket routes.
Production	<code>/production</code> , <code>/production/work-orders/:workOrderId</code>	Production board and Work Order Summary are separate concerns.
Approvals	<code>/approvals</code> , <code>/approvals/:approvalId</code>	Do not hide the approval system inside unrelated pages.

Documents	/documents, /documents/templates, /documents/questionnaires	DocuLink route group.
Inventory	/inventory, /inventory/:inventoryItemId	Purchasing is a business module, not an Orders sub-tab.
Purchasing	/purchasing, /purchasing/vendors, /purchasing/purchase-orders/:purchaseOrderId	Use a dedicated route group.
Pricing	/pricing, /pricing/calculator, /pricing/setup	One pricing module. Depreciated pricing pages must redirect or be removed.
Billing	/billing, /invoices, /invoices/:invoiceId	Billing owns customer invoice/payment controls; finance owns business-level analysis.
Finance	/financials, /financials/expenses, /financials/taxes	Avoid a second broad “Reports” nav destination.
Team	/team, /team/timesheets, /team/schedule, /team/payroll	Internal staff/admin workflow.
Productivity	/productivity, /productivity/calendar, /productivity/kanban, /productivity/tasks	One workspace with distinct views.
AI	/ai-tools, /ai-assistant, /ai-prompts, /ai-usage	Tools, assistant, prompts, and credits are related but separate pages.
Webstores	/webstores, /webstores/new, /webstores/:webstoreId	Store detail holds store-specific configuration tabs.
Wrap Lab	/wrap-lab or nested order-item route	Final decision: primary app entry may be /wrap-lab; wrap project detail remains tied to its Order Item.
Community	/community, /community/bugs, /community/feature-requests, /community/questions	Community is a single hub, not unrelated support pages.
Settings	/settings and approved subroutes	All settings-type pages consolidate here except platform admin settings.

2.4 Portal and Public Route Map

Area	Canonical Routes	Guard / Access
Customer Portal	/customer-portal/login, /customer-portal, /customer-portal/orders/:orderId, /customer-portal/proofs/:proofId, /customer-portal/invoices/:invoiceId, /customer-portal/documents/:documentId	Customer portal authentication or signed-access rules.
Employee Portal	/employee-portal/login, /employee-portal, /employee-portal/jobs/:orderId, /employee-portal/pay, /employee-portal/tasks, /employee-portal/profile	Employee authentication and employee record linkage.
Webstore Owner	/webstore-owner/onboard/:token, /owner-portal/signup/:token, /owner-portal	Token access for onboarding; authenticated access after account creation.
Public Storefront	/store/:storeSlug, /store/:storeSlug/cart, /store/:storeSlug/checkout	No internal navigation; public storefront guard and Stripe checkout behavior.
Public Forms	/forms/:publicRequestId, /questionnaires/:questionnaireId	Token/public access with expiration, anti-abuse, and response ownership checks.
Marketing	/, /features, /about, /contact, /support, /why-founder, /pricing, /privacy, /data-deletion	Public marketing/documentation routes only.

2.5 Route Guard Requirements

Guard	Applies To	Must Verify
RequireAuthenticatedUser	All non-public routes	Valid session/token and active user account.
RequireTenantMembership	Internal tenant routes	User belongs to current tenant and account is not suspended.
RequireRoleOrPermission	Role-limited internal routes	Explicit module permission, not a UI-only hide/show check.
RequireCustomerPortalAccess	Customer portal routes	Customer identity maps to the requested tenant/customer record.
RequireEmployeePortalAccess	Employee portal routes	User maps to active employee record for the tenant.

RequireOwnerPortalAccess	Owner portal routes	Owner/manager relationship exists for requested webstore.
RequirePlatformAdmin	Platform-admin routes	Platform-admin role independent of tenant staff roles.
RequirePublicTokenAccess	Tokenized forms/onboarding routes	Token valid, scoped, not expired, and still allowed to access the target record.

2.6 Legacy Route Migration Rules

8. Create a route inventory from the existing app before changing URLs. Mark every route as active, redirect, deprecated, hidden/test-only, or delete.
9. For a route with a clear canonical replacement, use a server/client redirect and preserve parameters where possible.
10. For a legacy Job Ticket route, redirect to the correct nested Order Item route only after the Order Item lookup has been verified.
11. For old pricing routes, redirect only if the destination provides the equivalent function. Otherwise leave a temporary explicit migration notice for internal users during testing, then remove it before launch.
12. Do not retain legacy routes just because bookmarks may exist. Record the redirect decision, expiration date, and test case.
13. Every redirect must have an automated test for a representative ID/parameter path.

2.7 Route Completion Checklist

- Each active screen is assigned one canonical route.
- Each route has an owner module and access zone.
- Each route has a guard decision documented.
- Each route appears in the approved navigation only where appropriate; no duplicate top-level nav items.
- All deep links load correctly after refresh.
- All legacy redirects are tested and have removal dates.
- There are no duplicate “Reports,” “Pricing,” “Admin,” or “Dashboard” destinations that overlap in purpose.

Step 3 - Tenant / Organization Ownership Rules

Objective: establish one reliable multi-tenant ownership model before data models and APIs are rebuilt. Tenant isolation is a backend responsibility. Every module must know which organization owns each record and which users may access it.

Critical Rule

A tenant ID must be applied and verified in backend reads, writes, updates, deletes, exports, file downloads, webhooks, and background jobs. A frontend filter is not security, it is merely decor.

3.1 Canonical Ownership Model

Tenant / Organization

- > has many Users through TenantMembership
- > has one or more Customers
- > has Orders, Order Items, Quotes, Invoices, Documents, Webstores, Employees, etc.
- > owns business settings and connected integrations unless explicitly platform-level

User

- > may belong to one or more Tenants through TenantMembership
- > may have internal role(s), platform-admin role, portal identity, or external owner relationship

TenantMembership

- > is the authoritative relationship for internal access
- > contains tenant_id, user_id, role(s), permission overrides, status, timestamps

3.2 Required Entity Ownership Fields

Entity Type	Required Ownership / Relationship Fields	Special Notes
Tenant-scoped core records	id, tenant_id, created_by_user_id, updated_by_user_id, created_at, updated_at	tenant_id must be indexed and required.
Customer-linked records	tenant_id, customer_id	Customer record itself also belongs to one tenant.
Order-linked records	tenant_id, order_id and where applicable order_item_id	Order Item remains a child of Order but has its own ID and status.
Portal identity/access records	tenant_id plus source relationship ID such as customer_id, employee_id, or webstore_owner_id	Never rely on email alone as the access relationship.
Files / attachments	tenant_id, storage_key, related_entity_type, related_entity_id, uploaded_by_user_id	All downloads must re-check relationship and permissions.
Settings	tenant_id or platform_scope; domain key; version; updated metadata	Use one explicit merge function for frontend configuration.
Activity logs	tenant_id, actor_user_id, event_type, entity_type, entity_id, payload, timestamp	Logs must be tenant-filtered by default.
AI usage	tenant_id, user_id, tool/action, credit_event_id, linked_entity_type/id	Needed for billing, audit, and support.
Webstore public records	tenant_id, webstore_id, owner relationship as applicable	Public slug lookup must still resolve to tenant-owned store.

3.3 Backend Tenant Scope Rules

- Resolve the authenticated actor and tenant context at the request boundary. Do not trust a tenant_id sent by the browser as proof of access.
- For internal routes, determine the tenant from the active tenant context plus a verified TenantMembership record.
- For record-specific routes, query by both record ID and tenant_id. Never query a tenant-scoped record by ID alone and filter it later.
- For create operations, set tenant_id on the server from verified context. The browser may suggest related records, but the server must validate they belong to the same tenant.

- For updates and deletes, load the record through a tenant-scoped query before applying changes.
- For bulk operations, add tenant filtering inside the query itself, then validate every selected record belongs to that same tenant.
- For background jobs, webhook handlers, exports, and file retrieval, pass explicit tenant context and revalidate record ownership.
- For cross-tenant platform-admin operations, require the platform-admin guard and log the action separately from tenant-user activity.

3.4 Roles and Access Relationships

Access Type	Authoritative Relationship	Examples
Internal tenant access	TenantMembership	Shop owner, admin, salesperson, production employee, staff.
Customer portal access	CustomerPortalAccess or customer-linked user identity	Customer sees only their own tenant/customer-scoped records.
Employee portal access	Employee record linked to user	Employee sees own schedule, time, tasks, assigned work, pay visibility defined by policy.
Webstore owner/manager access	WebstoreOwner / WebstoreManager relationship	Owner sees only assigned store(s), onboarding, approved sales/payout data.
Platform admin access	PlatformAdmin role/claim independent of tenant role	Can administer platform features; tenant impersonation must be logged.
Public token access	Scoped token record	A questionnaire, proof, owner onboarding link, or signed request access only to the target resource.

3.5 Tenant-Scoped Settings Architecture

The existing architecture notes correctly identify settings as a high-risk area. The frontend needs one merged configuration object, while the backend should store domain-specific settings in their correct domain services. Do not let every module invent its own giant unstructured settings blob.

Layer	Requirement
Backend domain storage	Store settings near the domain that owns them: pricing settings, payment settings, messaging settings, assistant settings, branding settings, webstore settings.
Tenant settings aggregator	Expose one explicit server-side merge function that assembles the approved tenant configuration shape for the frontend.
Frontend settings context	Consume the merged configuration object through one context/provider. It must not call scattered settings endpoints without a documented exception.
Write path	A write is routed to the owning domain service, validates tenant/permission rules, then invalidates/refetches the merged settings object.
Versioning	Each settings domain records update metadata and supports safe migrations when fields change.

3.6 Cross-Tenant Prohibitions

- No customer, order, document, invoice, employee, webstore, or file may be attached to a record from another tenant.
- No portal token may be accepted for a record outside the token's tenant and entity scope.
- No global search result may return tenant-scoped data outside the active tenant context.
- No report/export may query all tenants unless it is explicitly a platform-admin report.
- No integration credential may be exposed to another tenant. Tenant-level Stripe, email, or messaging settings must remain isolated.
- No application cache key may omit tenant identity when cached content depends on tenant data.

3.7 Required Tenant Isolation Tests

Test	Expected Result
Read record by valid ID from another tenant	Returns not found/forbidden without revealing record existence or details.
Update/delete record from another tenant	Fails before update/delete and creates no side effect.
Create child record using cross-tenant parent ID	Fails validation; no child record is created.
Download attachment from another tenant	Fails access check before a storage URL is issued.
Search while active in Tenant A	Never returns Tenant B records.
Portal user tries to modify another customer/employee/store record	Fails through relationship scope verification.
Platform admin impersonation	Works only with platform-admin privilege and produces a dedicated audit log event.
Background job/webhook	Uses supplied tenant scope and rejects mismatched related entity references.

3.8 Tenant Ownership Completion Checklist

- Every tenant-scoped model has tenant_id and an index or query plan appropriate to the datastore.
- Every protected API/service path uses centralized tenant scoping.
- Every create/update/delete operation validates related record ownership.
- Every portal access path has a separate relationship model and guard.
- Every file access path revalidates tenant/entity permissions.
- Settings are merged through one explicit server-side aggregator.
- Cross-tenant test cases are automated and passing.

Step 4 - Terminology Cleanup and Legacy Migration

Objective: define and enforce one vocabulary across UI labels, URLs, models, APIs, events, database fields, documents, tests, prompts, and developer documentation. The rebuild is the right time to stop carrying legacy terminology forward.

Canonical Rule

New code must use the final terminology immediately. Compatibility aliases may exist temporarily only for migration. Do not create new code that calls an Order Item a Job Ticket, even as a shortcut.

4.1 Canonical Vocabulary

Canonical Term	Definition	Use It In
Customer	A business or person receiving services, products, communications, documents, invoices, or portal access.	UI, models, APIs, reports, messaging, documents.
Quote	A proposed scope and price before commitment. A Quote may convert into an Order.	Quote pages, pricing, approvals, CRM, documents.
Order	The main committed customer transaction. An Order contains one or more Order Items.	Orders module, billing links, customer portal, production context.
Order Item	A specific product/service/work unit inside an Order. This replaces legacy Job Ticket terminology.	Nested routes, models, API endpoints, production linkage, pricing.
Work Order Summary	The production-facing summary generated from one or more Order Items.	Production board, print/export documents, employee workflow.
Artwork Proof	A customer-facing proof/version requiring review or approval.	Approvals, documents, customer portal, messaging.
Approval	A decision record tied to an Artwork Proof, document, quote, or other approval-capable item.	Approval module, audit logs, portals.
DocuLink	The document, questionnaire, template, attachment, and signature system.	Navigation label, modules, documentation, customer visibility rules.
Webstore	A tenant-owned or owner-managed online storefront that may create canonical Orders.	Internal app, portals, integrations, public storefront.
Wrap Lab	The specialized vehicle-wrap workflow module tied to vehicle-wrap Order Items.	Add-on branding, module folders, user-facing navigation.
Tenant / Organization	The shop/business account and data boundary.	Backend, access control, platform admin, settings.

4.2 Required Legacy-to-Canonical Mapping

Legacy / Problem Term	Replace With	Migration Action
Job Ticket	Order Item	Rename new UI and code immediately; legacy endpoint/route adapter only until all consumers move.
Job Tickets	Order Items	Update plural labels, navigation, API documentation, reports, and tests.
Job detail	Order Item detail	Rename route/page/component references; preserve legacy redirect only where necessary.
Production job	Work Order Summary or Order Item, depending on meaning	Audit each occurrence. Do not blindly replace where it actually means a production document.
Documents	DocuLink when referring to the system; Document when referring to one file/record	Update navigation and module documentation; avoid implying only file storage.
Wrap Command Center / WrapLab / Wrap Lab	Wrap Lab	Use one customer-facing name. If code needs a technical namespace, choose wrap-lab consistently.
PricingPage / PricingPagePublic	Pricing module pages	Remove duplicate/dead pages; retain only named calculator/setup/public onboarding pages where real.

Reports as a top-level catch-all	Financials, Reports, or module-specific analytics	Separate business finance from operational reports and platform analytics.
Admin Portal	Platform Admin or Internal Admin, depending on scope	Do not use “Admin” for both a tenant admin user and a platform administrator.

4.3 Terminology Rules by Technical Layer

Layer	Rule
UI labels	Use canonical user-facing term. Do not expose legacy labels except as a temporary migration note for internal beta users.
Routes	Use canonical nouns. Example: /orders/:orderId/items/:orderId, not /job-tickets/:id.
Frontend component names	Use canonical names. Example: OrderItemDetailPage, not JobTicketDetail.
Backend endpoint names	Use canonical resource names. Example: /api/order-items, not /api/job-tickets.
Database models	Create canonical model names and fields. Avoid a permanent duplicate model solely to preserve a legacy name.
Event/activity names	Use names such as order_item.created, order_item.status_changed, work_order.generated.
Tests	Test canonical behavior and routes. Compatibility tests should be grouped as legacy migration tests.
Prompt/documentation text	Use canonical terms so future AI tools, support docs, and builders learn the correct structure.

4.4 Migration Strategy

- Create a terminology inventory by searching the repository, database schemas, API routes, documentation, tests, translations, prompts, email templates, and generated document templates.
- Classify every legacy occurrence as one of: direct rename, semantic split required, compatibility adapter, historical record label, or delete.
- Perform model and API design updates before broad UI wording updates. The backend contract must be clear first.
- Create canonical routes/endpoints and update all active internal callers to use them.
- Add temporary legacy redirects/adapters only for external bookmarks, deployed clients, or migration-critical integrations. Each must include an owner and removal date.
- Migrate historical data carefully. A legacy Job Ticket record should map to an Order Item only after the parent Order relationship and all child references are confirmed.
- Update generated documents and portal language after core data/API changes so customer-facing terms match the new system.
- Run a final repository search that must return zero newly-authored legacy terms outside approved migration adapters, historical migration scripts, and explicitly preserved audit history.

4.5 Semantic Split Rules: Do Not Use Blind Find-and-Replace

Important

Some old terms may represent different concepts in different files. “Job Ticket” may mean an Order Item in one location, a production sheet in another, or a legacy route ID in a third. Each occurrence must be classified before renaming.

When Legacy Term Means...	Replace With...	Example
A sellable/requested work unit within an Order	Order Item	Vehicle wrap, banner, sign panel, install service within one Order.
A production instruction sheet created from work data	Work Order Summary	Internal production packet/sheet used by the shop floor.
A historical database identifier or import field	Legacy mapping field with documentation	Keep only in migration layer/historical record compatibility.
A user-facing page that is now obsolete	Canonical replacement route or removal	Redirect old /jobtickets/:id to /orders/:orderId/items/:orderId when mapping exists.

4.6 Required Naming Conventions

Item	Convention	Example
Folder names	lowercase kebab-case	wrap-lab, inventory-purchasing, finance-billing
Component/page names	PascalCase canonical business name	OrderItemDetailPage.tsx
API route segments	lowercase kebab-case plural resources	/api/order-items
Database entity/model names	PascalCase singular canonical name	OrderItem
Database collection/table names	lowercase plural or existing datastore convention, consistent across models	order_items or orderItems - choose one style and enforce it
Event names	dot notation, canonical nouns	order_item.status_changed
Status values	lowercase snake_case or approved enum style	awaiting_approval, in_production
IDs	Explicit semantic ID fields in DTOs and parameters	orderId, orderItemId, tenantId

4.7 Terminology Cleanup Completion Checklist

- Canonical vocabulary is approved and stored in docs/architecture/terminology.md.
- New model, component, route, and API names use canonical terms.
- Every legacy Job Ticket occurrence is classified and has a migration decision.
- No active navigation label uses legacy terminology.
- No newly written user-facing template, email, prompt, or PDF uses deprecated labels.
- Legacy aliases are isolated, documented, tested, and assigned a removal date.
- Repository search confirms deprecated terms occur only in approved historical/migration locations.

Implementation Handoff for Emergent

Use this instruction when assigning Phase 0 Steps 1-4. It intentionally limits the work to architecture, migration planning, and controlled refactoring. It does not authorize rebuilding unrelated business features.

You are implementing Phase 0, Steps 1-4 of the SignGuy AI rebuild foundation:

1) final folder structure, 2) final route structure, 3) tenant/organization ownership rules, and 4) terminology cleanup.

Work only from the approved Phase 0 Master Foundation Specification. Do not build new business features.

Required behavior:

- First produce an implementation plan listing exact affected files, move/rename decisions, route changes, schema/API changes, risks, and tests.
- Do not guess. Mark uncertain current behavior as Needs Verification.
- Preserve working behavior while introducing canonical structure and temporary compatibility adapters only where needed.
- Do not introduce new duplicate contexts, pages, models, routes, or settings stores.
- Enforce tenant scoping in backend read/write/update/delete/export/file-access paths.
- Use canonical terminology: Quote -> Order -> Order Item(s) -> Work Order Summary. New code must not introduce Job Ticket terminology.
- Create a migration log in docs/architecture/phase-0-migration-log.md.
- After each step, run the required tests and provide a concise completion report with: files changed, routes affected, compatibility adapters created, tests passed, and unresolved decisions.

Stop after Step 4 and wait for approval before proceeding to Steps 5-10 or rebuilding business modules.

Phase 0 Deliverables Checklist

Deliverable	Required Contents
Target Folder Map	Final tree, module ownership rules, shared-code criteria, legacy quarantine policy.
Route Registry	Canonical routes, access zone, owner module, guard, redirect/deprecation status.
Tenant Ownership Standard	Entity ownership requirements, backend tenant scope middleware/service pattern, portal relationship rules, settings merge pattern.
Terminology Glossary	Canonical terms, legacy mappings, semantic split notes, code naming conventions.
Migration Log	Each moved/renamed/deprecated item, reason, compatibility action, removal date, test status.
Test Evidence	Import/build checks, route checks, tenant isolation tests, redirect tests, terminology scan results.

Owner Decisions Still Required Before Implementation

- Confirm the final customer-facing add-on name: “Wrap Lab” is recommended as the single branded name.
- Confirm whether Wrap Lab should have both a top-level internal route (/wrap-lab) and a nested Order Item entry, or only one canonical entry with contextual links.
- Confirm the preferred database naming convention: snake_case collections/tables versus existing camelCase convention. Choose one and apply consistently in new code.
- Confirm whether tenant users may belong to multiple shop tenants in the first rebuild release. The architecture should support it even if the UI initially does not expose switching.
- Confirm which legacy routes must remain temporarily accessible for deployed users/bookmarks, and the intended removal window.

Reference Basis

This Phase 0 specification was prepared from the current SignGuy AI app architecture audit, including its module inventory, route map, recommended module-oriented folder structure, dependency map, tenant/shared-system notes, duplicate/legacy findings, and recommended final navigation structure.

Important Reminder

Phase 0 is done when the rules are implemented and proven, not when a new folder tree looks tidy. The test evidence, migration log, tenant-scoping checks, route checks, and legacy-term scan are the actual proof of completion.

SIGNGUY AI

Phase 0 Master Foundation

Part 2: Shared Patterns

Detailed implementation instructions for Steps 5-7: shared event and activity logging, settings merge architecture, and permissions

Purpose

This is the second Phase 0 implementation document. It defines the three cross-cutting patterns that every rebuilt module must reuse: the event and activity log pattern, the settings merge pattern, and the permissions pattern. These are shared systems, not optional “nice later” features.

Document Control	Value
Scope	Phase 0, Steps 5-7 only
Applies To	All frontend modules, backend services, database records, portals, integrations, automations, and add-ons
Predecessor	Phase 0 Part 1: folder structure, routes, tenant ownership, terminology cleanup
Primary Architectural Principle	Build shared infrastructure once; module teams consume it rather than inventing replacements.
Out of Scope	Building final dashboards, module-specific workflows, reporting screens, AI interfaces, webstore UI, or Wrap Lab UI

How This Document Fits Phase 0

Steps 5-7 sit directly on top of the first four Phase 0 decisions. Folder ownership tells you where the code belongs. Route structure tells you which access zone a user is in. Tenant rules tell you which organization owns a record. This document tells you how the system records actions, resolves configuration, and decides what a user may do.

Step	Shared Pattern	Mandatory Outcome
5	Event and activity log pattern	Every meaningful state change produces a consistent, queryable, tenant-scoped event.
6	Settings merge pattern	The frontend receives one stable resolved settings object while backend settings stay domain-owned.
7	Permissions pattern	Every access decision is enforced consistently in the backend and mirrored in the frontend for usability.

Non-Negotiable

A module may add module-specific event types, settings fields, or permission capabilities. It may not create a private activity-log format, a second all-purpose settings object, or one-off role checks scattered through components.

Step 5 - Shared Event and Activity Log Pattern

Objective: establish one reliable event system that records meaningful business activity, supports user-facing timelines, powers alerts and automation later, and creates a trustworthy audit trail. The current architecture audit identifies fragmented event handling and several activity/audit log concepts. The rebuild must define one canonical event envelope and clear visibility rules.

5.1 What This System Must Do

- Create an immutable event record whenever a meaningful business action or system action occurs.
- Allow the same event stream to power a record timeline, a customer timeline, an order timeline, a user activity feed, notifications, automation triggers, reports, and platform audit views without duplicating data.
- Distinguish human actions, automated system actions, integration actions, and AI actions.
- Capture enough “before” and “after” information to explain what changed without dumping sensitive entire records into the log.
- Keep all tenant-scoped events isolated to the correct organization, while allowing platform-level audit events for platform administrators.
- Support chronological display, filtering, debugging, and retention rules without becoming the database for every document version or message body.

5.2 One Canonical Event Envelope

Use a single Event / Activity record shape across the application. Module-specific details go in metadata, but the identity, tenant, actor, target, type, time, visibility, and correlation fields must be consistent.

Field	Required	Purpose / Rule
id	Yes	Immutable unique event ID.
tenant_id	Yes for tenant events	Organization owner. Must be set by backend context, never trusted from frontend input.
event_type	Yes	Namespaced action, for example <code>order.created</code> or <code>proof.approved</code> .
event_category	Yes	Human-readable family: customer, quote, order, production, payment, document, message, system, security, AI, integration.
occurred_at	Yes	Server-generated UTC timestamp.
actor_type	Yes	user, customer, employee, system, integration, AI, platform_admin.
actor_id	Conditional	ID of the actor when applicable. System events may use null with <code>actor_type=system</code> .
actor_display	Yes	Snapshot display name/email safe for timeline rendering after a user is renamed.
subject_type	Yes	Primary record type affected, such as order, customer, invoice, document.
subject_id	Yes	Primary record ID affected.
related_entities	No	Small structured list of linked records, e.g. customer, order, order_item, invoice.
action	Yes	Past-tense readable action phrase, e.g. “sent proof for approval.”
before	Conditional	Only changed fields needed for audit or recovery. Never store full sensitive objects.
after	Conditional	Only changed fields needed for audit or recovery.
metadata	No	Module-specific structured data. Validate per event type.
visibility	Yes	internal, customer_portal, employee_portal, owner_portal, platform_admin, private_admin.
severity	Yes	info, success, warning, critical.
correlation_id	Yes	One ID carried across a multi-step workflow, request, webhook, or automation.
request_id	Recommended	Connects API request logs/errors to a business event.
source	Yes	ui, api, webhook, scheduled_job, import, AI, migration.
version	Yes	Event-schema version for future

		compatibility.
--	--	----------------

```
{
  "event_type": "order.status_changed",
  "tenant_id": "tenant_123",
  "actor_type": "user",
  "actor_id": "user_456",
  "subject_type": "order",
  "subject_id": "order_789",
  "action": "changed order status from Artwork Pending to In Production",
  "before": { "status": "artwork_pending" },
  "after": { "status": "in_production" },
  "related_entities": [{ "type": "customer", "id": "customer_12" }],
  "visibility": "internal",
  "correlation_id": "corr_abc",
  "source": "ui",
  "version": 1
}
```

5.3 Event Naming Convention

Use lower-case dot notation: <module>.<noun_or_state>.<verb>. Keep names stable and explicit. Do not use vague events such as updated, changed, or action_completed with no module context.

Good	Avoid	Reason
customer.created	created	Searchable and module-owned.
quote.converted_to_order	quote.done	Preserves the business transition.
order.status_changed	order.updated	States what changed.
proof.approved	approval.success	Names the actual business entity.
invoice.payment_received	payment.updated	Separates payment receipt from other invoice changes.
document.sent_to_customer	document.action	Useful for portal, email, and audit timelines.
ai.quote_draft_generated	assistant.used	Explains what AI did and supports credit usage tracing.

5.4 Required Event Families

Family	Minimum Required Events
Security & access	auth.login_succeeded, auth.login_failed, auth.password_reset_requested, auth.password_changed, auth.account_suspended, permission.denied
Tenant & user	tenant.created, tenant.settings_changed, user.created, user.invited, user.role_changed, user.deactivated
Customer & CRM	customer.created, customer.updated, customer.merged, lead.created, lead.converted
Quotes & orders	quote.created, quote.sent, quote.approved, quote.converted_to_order, order.created, order.status_changed, order_item.created, order_item.updated
Production & approvals	work_order.created, production.stage_changed, proof.created, proof.sent, proof.approved, proof.revision_requested
Finance	invoice.created, invoice.sent, invoice.payment_received, invoice.payment_failed, payment.refunded, expense.created
Documents & forms	document.uploaded, document.generated, document.sent, document.signed, questionnaire.sent, questionnaire.completed
Messaging	message.sent, message.delivered, message.failed, conversation.created, notification.created
Inventory & purchasing	inventory.adjusted, inventory.received, purchase_order.created, purchase_order.received
Webstores	webstore.created, webstore.launches, webstore_order.created, stripe_connect.completed, payout.status_changed
AI	ai.requested, ai.completed, ai.failed, ai.credit_debited, ai.credit_refunded
System & integrations	webhook.received, webhook.failed, import.completed, scheduled_job.failed, migration.completed

5.5 What Must Create an Event

- Creation, deletion, archive, restore, merge, status/stage change, ownership change, assignment change, approval/rejection, signature, payment change, and pricing override.
- Any customer-facing communication sent or failed, including email, SMS, portal message, proof request, document request, appointment reminder, and payment reminder.
- Any role, permission, tenant, integration credential, Stripe connection, or platform-admin action that affects access, money, or configuration.
- Every AI request, result, failure, credit debit, credit refund, or action carried out on a record.
- Any automation or webhook that changes business data. The actor must be system or integration, not a fake human user.

Do Not Log Noise

Do not create timeline entries for every page view, keystroke, dropdown open, autosave heartbeat, list refresh, or harmless UI event. Analytics can collect usage separately. The activity log is for business meaning, traceability, and action history.

5.6 Write Path and Service Rule

Events must be created by backend service methods, inside the transaction or immediately after the transaction commits. Frontend components must never write activity events directly. This prevents forged audit history and keeps the event tied to the real database outcome.

1. Frontend submits an intent to an authenticated API endpoint.
2. Backend resolves tenant, actor, permissions, and request/correlation IDs.
3. Module service validates and updates the primary record.
4. Service generates an event from approved server-side data.
5. Transaction commits. If the transaction fails, no success event is written.
6. Optional event dispatcher sends notifications, automation, search indexing, or analytics downstream after commit.

```
await orderService.changeStatus({ tenantId, actor, orderId, nextStatus, correlationId })
-> validate permission and allowed transition
-> update Order
-> append ActivityEvent(order.status_changed)
-> commit
-> dispatch non-critical subscribers asynchronously
```

5.7 Timeline and Visibility Rules

Visibility	Who Can See It	Typical Use
internal	Authorized shop users with record access	Order stage changes, internal production notes, internal assignments.
customer_portal	Customer plus authorized staff	Proof sent, invoice available, document signed, appointment scheduled.
employee_portal	Assigned employee plus authorized admins	Clock events, assigned work, schedule changes.
owner_portal	Webstore owner/manager plus authorized staff	Store launched, payout status, owner approval request.
platform_admin	Platform admins only	Tenant impersonation, platform configuration, suspicious activity.
private_admin	Restricted administrative roles only	Sensitive staff, financial, disciplinary, or security operations.

5.8 Event Data Safety Rules

- Never store passwords, tokens, Stripe secrets, full payment card data, raw access links, or sensitive authentication payloads in events.
- Never dump entire customer records, full document contents, message bodies, or large file data into before/after fields.
- Use IDs plus concise snapshots. For example, record document_id and document title, not a PDF blob.

- For sensitive fields, log that a value changed without logging the original or new secret. Example: “payment account updated.”
- Use metadata validation per event type. Unknown arbitrary blobs become unsearchable junk very quickly, which is apparently a favorite software tradition.

5.9 Event Query and Retention Requirements

Requirement	Implementation Rule
Record timeline	Query by tenant_id + subject_type + subject_id, sorted occurred_at descending.
Customer history	Query events that reference the customer in subject or related_entities.
Order history	Query all events referencing the order and its order items, optionally grouped by date.
System audit	Filter by event_type, actor, source, severity, date range, correlation_id.
Indexes	At minimum: tenant_id+occurred_at, tenant_id+subject_type+subject_id+occurred_at, tenant_id+actor_id+occurred_at, correlation_id, event_type+occurred_at.
Retention	Keep business and financial audit events according to product/legal requirements. Do not casually purge them because the table got large.
Archive	Older events may move to archival storage only if timelines and audits can still retrieve them when needed.

5.10 Step 5 Required Deliverables

- Canonical ActivityEvent / DomainEvent schema and migration.
- Event type registry with documented valid event names and metadata schema per family.
- Backend event writer service used by all new module services.
- Correlation/request ID middleware or equivalent request-context utility.
- Reusable timeline query service and a basic internal timeline component.
- Visibility enforcement on event reads.
- Automated tests proving events are written on successful state changes and not written on failed transactions.
- Migration strategy for existing audit/activity records, including whether they are imported, mapped, archived, or left read-only in the legacy app.

Step 6 - Shared Settings Merge Pattern

Objective: create one predictable settings architecture. The frontend should receive a resolved settings object for convenient display and behavior, while the backend keeps domain-specific settings in the domain that owns them. The architecture audit specifically warns against scattered merge logic and the old {id} versus {tenant_id} confusion. This step fixes that at the foundation.

6.1 Core Rule

Single Resolver, Not a Single Junk Drawer

There may be many settings stores, but there must be one explicit backend settings resolver that returns the resolved settings object for a tenant. Do not create one giant unstructured tenant_settings JSON object that every module edits freely. That is a landfill with a save button.

6.2 Ownership Model

Settings Domain	Owned By	Examples	Storage Rule
Organization / company	Tenant module	Shop name, address, logo, timezone, currency, business hours	Tenant-owned record or organization settings document.
Branding	Branding/settings module	Logo, colors, email sender branding, portal appearance	Separate branding domain, merged into resolved app settings.
Pricing	Pricing module	Shop rate, markup defaults, labor rates, material defaults, tax defaults	Pricing-owned schema. No direct editing from generic settings object.
Documents & templates	DocuLink / template module	Document templates, email templates, signature defaults	Module-owned records, referenced by settings resolver when needed.
Notifications	Messaging module	Email/SMS provider config references, notification preferences, reminder rules	Module-owned and permission-restricted.
Integrations	Integration module	Stripe connection IDs, SendGrid references, Meta configuration, webhook preferences	Separate secure integration configuration; secrets never returned to frontend.
AI	AI platform module	Default model behavior, credit rules, assistant preferences, allowed tools	AI-owned schema.
Production	Production module	Stages, default queues, production rules	Production-owned schema.
User preferences	User module	Theme, density, default landing page, notification preferences	User-scoped, never written to tenant settings.
Feature access / entitlements	Subscription/platform module	Plan, enabled add-ons, credit limits, experimental flags	Platform-controlled; tenant cannot self-edit entitlement fields.

6.3 Resolved Settings Object

The frontend may call one endpoint such as GET /api/settings/resolved. That endpoint must compose safe, tenant-scoped, permission-aware values from the owning domains. It is a read model, not the universal write target.

```
{
  "tenant": { "id": "tenant_123", "name": "Sign Guy PA", "timezone": "America/New_York" },
  "branding": { "logo_url": "...", "theme": { "accent": "#..." } },
  "pricing": { "shop_rate": 85, "currency": "USD" },
  "production": { "stages": ["artwork", "production", "install"] },
  "features": { "webstores": true, "wrap_lab": true, "ai_assistant": true },
  "user_preferences": { "theme": "light", "default_landing": "/dashboard" },
  "permissions": { "can_manage_pricing": true, "can_view_payroll": false },
  "revision": "settingsrev_2026_07_02_001"
}
```

6.4 Required Resolver Behavior

- Resolve the tenant from authenticated backend context, not from a client-provided tenant_id.
- Load base tenant settings first, then module-owned domain settings, then plan/entitlement gates, then user-specific preferences.
- Apply a defined precedence order. Higher-precedence values may override lower ones only where the schema explicitly allows it.
- Remove secrets, internal provider credentials, raw webhook signing keys, hidden feature flags, and restricted financial configuration before responding.
- Attach a revision/version identifier so the client can cache and invalidate safely.
- Fail clearly by domain. If AI settings are unavailable, the resolver should degrade that domain rather than making the entire app settings endpoint fail.
- Be the only merge implementation. No page, route, React context, or random backend controller may hand-merge settings from multiple sources.

6.5 Settings Precedence Order

Priority	Layer	Allowed To Override
1 - Highest	Platform security and compliance rules	May restrict any lower-level setting. Tenant cannot override.
2	Subscription / entitlement rules	Controls available modules, quotas, and plan-locked behavior.
3	Tenant organization defaults	Company-wide operational defaults.
4	Module domain settings	Only their own named domain and declared integration points.
5	Role / team defaults	Default behavior for a role where allowed.
6 - Lowest	User preferences	Presentation and personal workflow preferences only; never access, pricing, tax, or security rules.

6.6 Read and Write Rules

Action	Correct Pattern	Never Do This
Read resolved settings	GET /api/settings/resolved through resolver service	Frontend queries five stores then improvises a merge.
Update company profile	PATCH /api/settings/company -> company settings service	PATCH a giant /settings blob with unrelated fields.
Update pricing defaults	PATCH /api/pricing/settings -> pricing service	Allow generic settings page to write pricing object directly.
Update user theme	PATCH /api/users/me/preferences -> user service	Store personal theme on tenant record.
Update Stripe config	Integration-specific secure endpoint	Return secret keys back to the browser.
Refresh after update	Server returns updated domain revision; client invalidates resolved settings cache	Mutate a stale local settings object and hope other pages notice.

6.7 Schema and Validation Rules

- Every settings domain needs a typed schema, defaults, validation, and explicit migration version.
- Use defaults at the resolver/domain-service level, not scattered fallback values in UI components.
- No settings field may be silently ignored. Reject unknown fields or report validation errors clearly.
- Every settings update requires tenant context, authorization, schema validation, event logging, and revision increment.
- Domain services must validate cross-field rules. Example: a payment reminder cannot be enabled without an approved sender configuration.
- Config changes that affect money, access, integration behavior, or customer communication must generate a settings-changed event with safe before/after summaries.

6.8 Caching and Invalidation

Scenario	Required Behavior
----------	-------------------

Normal page load	Use cached resolved settings with revision/ETag validation where practical.
User changes own preference	Refresh user preference segment immediately; do not force a full tenant settings rebuild unless required.
Admin changes tenant setting	Increment relevant domain and resolved revision; invalidate affected client caches.
Entitlement changes	Invalidate features/permissions segment and hide unavailable module entry points immediately.
Integration reconnect	Refresh only integration-safe status fields; never expose credentials.
Failed settings update	Keep prior resolved revision active; show a useful error and do not partially mutate client state.

6.9 Required Settings Domains for the Rebuild

Domain	Must Be Available In Early Rebuild	Notes
company	Yes	Company identity, timezone, address, contact basics.
branding	Yes	App/portal logo and basic theme identity.
pricing	Yes	Required before quote/order workflows.
production	Yes	Required when production stages are rebuilt.
documents	Yes	Templates and signature defaults as DocuLink begins.
notifications	Yes	Sender configuration, notification preferences, reminder behavior.
integrations	Foundation only	Connection status and safe IDs first; full integrations can follow.
AI	Foundation only	Credit/feature gates before AI tools are built.
user_preferences	Yes	Theme/density/default start page.
features_entitlements	Yes	Controls add-ons and plan access.

6.10 Step 6 Required Deliverables

- Typed schemas for tenant/company, branding, pricing, notifications, integrations, AI, production, user preferences, and entitlements.
- One settings resolver service with documented precedence and safe response shaping.
- One resolved settings endpoint plus domain-specific write endpoints.
- Revision/invalidation approach and client settings provider that consumes only the resolved endpoint.
- Permission checks on every write endpoint.
- Event generation for security-sensitive or business-sensitive settings changes.
- Tests for tenant separation, precedence, forbidden secret exposure, invalid payload rejection, cache revision change, and domain failure behavior.
- Migration map from legacy settings screens/routes to the new owning domains.

Step 7 - Shared Permissions Pattern

Objective: establish one permission system that is understandable, tenant-safe, backend-enforced, portal-aware, and reusable across every module. The system must support the app's internal users, customer portal users, employee portal users, webstore owners/managers, and platform admins without letting each module invent its own role checks.

7.1 Core Rules

- Backend enforcement is the source of truth. Frontend controls may hide unavailable actions, but they never provide security.
- Roles group permissions for convenience. Permissions, not hard-coded role names, determine access decisions.
- All permission checks operate inside an access zone and tenant context.
- A user may have different roles in different tenants. Do not treat a global user role as sufficient for tenant-scoped access.
- Portal users are not internal staff. Their allowed actions must be explicitly scoped to their own customer, employee, or webstore-owner relationship.
- Platform administrators operate in a separate platform-admin zone and do not implicitly become tenant staff unless they use controlled, audited impersonation or support access.

Do Not Scatter Role Checks

Do not write dozens of checks such as `if (user.role === "admin")` throughout pages and services. Use capability checks such as `can("orders.edit", order)` so the rule can evolve without rebuilding every screen.

7.2 Access Zones

Access Zone	Who Uses It	Examples	Boundary Rule
Internal tenant app	Shop owner, tenant admin, staff, salesperson, production users	/dashboard, /orders, /customers, /financials	Requires tenant membership and internal-app permissions.
Customer portal	Customers/contacts	/customer-portal/*	May only access records explicitly linked to their customer/contact identity.
Employee portal	Employees	/employee-portal/*	May only access own time/pay/profile and assigned work unless extra permission is granted.
Webstore owner/manager portal	Store owners and delegated managers	/owner-portal/*	May only access their assigned webstore(s), payouts, catalog, approvals, and store records.
Public	Unauthenticated visitors	/store/:slug, public forms, marketing pages	No internal data. Public tokens/slug must be rate-limited and scoped.
Platform admin	Platform staff only	/platform-admin/*	Separate role namespace, MFA/security controls when introduced, full audit logging.

7.3 Permission Naming

Use resource.action names. Add scope only when it clarifies the rule. Keep names stable, module-owned, and easy to read.

Permission	Meaning
customers.view	View customers allowed by scope.
customers.create	Create customer records.
customers.edit	Edit customer records.
customers.merge	Merge duplicate customer records.
quotes.convert	Convert an approved quote into an order.
orders.view	View eligible orders.
orders.edit	Edit eligible orders.
orders.change_status	Change order workflow status.
orders.view_financials	View costs, margin, or internal financial data attached to an order.
production.assign	Assign production work.
proofs.send	Send proof approval request.
proofs.approve_internal	Mark internal approval where that exists.
invoices.manage	Create/edit/send invoices.
payments.refund	Issue or initiate refund operation.

inventory.adjust	Adjust inventory quantities.
payroll.view	View payroll data.
payroll.manage	Create/edit payroll transactions.
settings.manage_pricing	Edit pricing settings.
settings.manage_integrations	Edit integrations.
webstores.manage	Manage assigned webstores.
webstores.view_payouts	View webstore payout information.
ai.use	Run permitted AI actions.
platform.tenants.manage	Manage platform tenants.

7.4 Scope Model

A permission alone is not enough. The scope determines which records the actor can act on. Every authorization decision must evaluate both the capability and the record relationship.

Scope	Meaning	Example
all_tenant	All records in current tenant	Owner can view all orders in their shop.
team	Records assigned to user's team or department	Production lead sees production work for their team.
assigned	Records explicitly assigned to the user	Installer sees their assigned work orders.
own	Records created by or owned by the actor	Employee sees own timesheet; customer sees own portal records.
related_customer	Records linked to the customer contact	Customer can see invoices and proofs for their business.
assigned_webstore	Records tied to a specific owner/manager webstore	Store owner sees their store catalog and payout status.
platform	Cross-tenant platform records	Platform admin manages tenants, not tenant records by default.

7.5 Standard Roles for Initial Rebuild

Role	Primary Scope	Starting Capability Intent
Tenant Owner	all_tenant	Full tenant control except immutable platform rules.
Tenant Admin	all_tenant with exclusions	Operational administration; payroll/integrations may be separately granted.
Sales / Customer Service	team or all_tenant	Customers, quotes, selected orders, messaging, proofs; no payroll or restricted financial configuration.
Production Manager	team or all_tenant	Orders, order items, production, proofs, inventory usage, scheduling.
Production Employee	assigned	Assigned work, production status updates, own tasks; restricted costs and customer financials.
Installer	assigned	Assigned installs, appointment details, completion photos/checklists; restricted pricing/payroll.
Accounting / Finance	all_tenant financial	Invoices, payments, expenses, taxes, financial reports; not production admin by default.
Payroll Manager	all_tenant payroll	Timesheets, payroll, schedule; restricted customer financials unless additionally granted.
Customer Portal User	related_customer	Own customer records, documents, proofs, invoices, payments, messages, forms.
Employee Portal User	own / assigned	Own time, pay, schedule, profile, assigned jobs/tasks.
Webstore Owner	assigned_webstore	Own store dashboard, approval, Stripe/payout status, limited store analytics.
Webstore Manager	assigned_webstore	Catalog, products, store setup, orders, owner collaboration; payout scope explicit.
Platform Admin	platform	Platform management only; tenant impersonation is explicit and audited.

7.6 Authorization Flow

7. Authenticate the actor and establish identity/session.
8. Resolve access zone from route/API endpoint.
9. Resolve tenant from host, route, membership, public token, or portal relationship.
10. Load role grants and direct grants for that tenant/access zone.
11. Load the target record when authorization depends on ownership/assignment relationship.
12. Evaluate capability + scope + record relationship + state-based rule.
13. Allow or deny. Denials must return a safe error and write security/audit telemetry when appropriate.

```
authorize({ actor, tenantId, zone: "internal", capability: "orders.edit", resource: order })
  -> actor has orders.edit in tenant_123
  -> scope permits all_tenant OR order.assigned_user_id == actor.id
  -> order is not locked/finalized for this action
  -> allow / deny
```

7.7 State-Based Permissions

Some actions depend on record state, not only role. Put these rules in the module service or policy layer, never only in UI button visibility.

Example	Required State Rule
Quote conversion	Only a valid, non-archived quote may convert to an order.
Proof approval	Customer can approve only a proof sent to that customer and not superseded. Staff may override only with explicit permission and reason event.
Invoice modification	Invoice edits may be limited after payment is received; credit/refund workflow may be required.
Payroll finalization	Finalized pay period changes require explicit reopen permission and audit event.
Webstore payout access	Owner may view only payout data for their connected store account.
Document download	Portal user may download only documents marked visible to their relationship and still available.

7.8 Frontend Permission Use

- Load a safe permission summary in the resolved settings/session payload for navigation and action visibility.
- Use a shared can(capability, resource?) helper in components and route guards.
- Hide unavailable actions and explain read-only states where useful, but do not rely on hidden controls for security.
- Never ship raw role logic through every page. UI should ask “can this user edit this order?” rather than comparing role names.
- When API denies an action, show a human-readable message without revealing hidden record data or security rules.

7.9 Backend Enforcement Requirements

- Every protected API endpoint must identify its required capability and access zone.
- Every endpoint acting on a tenant record must enforce tenant membership/relationship and record scope in the database query or policy layer.
- Never query by record ID alone and check tenant after returning the record. Scope the query before data is loaded or exposed.
- Public token routes must use token-scoped access and expiration rules; a public token is not an internal login.
- Platform admin endpoints require platform permissions. Impersonation must create a platform audit event and clearly mark the acting identity.
- Permission failures for sensitive actions should be logged with actor, attempted capability, zone, route, target type/ID, and correlation ID, without exposing secret input data.

7.10 Permission Administration

Administration Need	Rule
Role templates	Provide standard roles above as editable starting templates, not hard-coded forever.
Custom roles	Allow later, but use named capability bundles and validation. Do not

	launch arbitrary role editing until the core capability registry exists.
Direct grants	Use sparingly for exceptions. Always show who granted it, why, and expiration if applicable.
Role changes	Require authorized admin, create role-changed event, invalidate permissions/session cache.
Deactivation	Remove active membership/access immediately; preserve historic event actor snapshots.
Review	Provide a future permission review screen showing user, roles, scopes, sensitive grants, and last changed date.

7.11 Step 7 Required Deliverables

- Capability registry grouped by module, including descriptions and access-zone applicability.
- Role templates and tenant membership model.
- Authorization/policy service with capability + scope + record-state evaluation.
- Backend middleware/decorator pattern for protected endpoints.
- Frontend can() helper, safe permission summary, and route/action guard components.
- Portal-specific policy rules for customer, employee, and webstore owner/manager relationships.
- Platform-admin namespace and audited impersonation/support-access design.
- Tests for cross-tenant denial, portal relationship denial, role-change invalidation, state-based denial, and sensitive data restrictions.

Steps 5-7 Combined Verification Plan

Do not declare Phase 0 Part 2 finished because code compiles. These patterns are foundational specifically because bugs in them spread to every module. Verify them before Orders, Webstores, Wrap Lab, payroll, and portals are rebuilt.

Test Area	Minimum Proof of Completion
Event writing	Create/update/status-change actions produce correct events only after successful transaction commit.
Event isolation	Tenant A cannot query Tenant B timeline events; customer portal cannot see internal-only timeline events.
Event safety	Secrets and full sensitive payloads never appear in event records.
Settings resolve	Different tenant settings resolve correctly; user preference affects only that user; secret fields are absent.
Settings writes	Domain-specific endpoint validates, authorizes, increments revision, writes event, and refreshes correct client data.
Permission enforcement	Direct API calls cannot bypass UI-hidden actions. Cross-tenant record IDs return not found or forbidden safely.
Portal scope	Customer, employee, and webstore portal users can access only their linked records.
Platform boundary	Platform admins cannot silently act as tenant staff; impersonation/support access is explicit and logged.

Builder Handoff Prompt for Phase 0 Part 2

Copy This Into Emergent

Implement Phase 0 Part 2 of the SignGuy AI rebuild exactly as specified in the attached document. Complete only Steps 5-7: (5) the canonical shared event/activity-log pattern, (6) the domain-owned settings resolver and merge pattern, and (7) the shared capability-based permissions pattern. Do not build unrelated business modules. First inspect the current repository and identify existing event logs, audit logs, activity records, settings objects, role checks, portal guards, and tenant middleware. Produce an implementation plan that maps existing code to the new patterns before editing. Then implement in small verifiable commits. Preserve tenant isolation, enforce permissions in the backend, avoid new scattered role checks, do not expose secrets through resolved settings, and use canonical event names. End with a change log, route/API list, schema changes, migration notes, tests run, test results, and unresolved decisions.

Phase 0 Part 2 Exit Gate

- One canonical activity/event schema, registry, writer service, and timeline read model exist.
- All new module writes can produce tenant-scoped events through the shared service.
- One documented settings resolver returns a safe resolved settings object with clear precedence.
- All settings writes are domain-owned, validated, authorized, versioned, and event-logged.
- One capability-based permission model exists across internal app, portals, public access, and platform admin.
- Backend authorization is proven with automated tests, including cross-tenant and portal-scope failures.
- No new module work begins until the builder delivers the implementation report and unresolved policy decisions are reviewed.

Owner Decisions Still Needed Before Later Phases

Decision	Why It Matters	Recommended Default
Custom roles at launch?	Affects admin UI and data model complexity.	Launch standard role templates plus limited custom grants; build full custom role editor later.
Activity event retention period	Affects storage and compliance behavior.	Keep operational/financial/security events long-term; archive rather than delete.
Can staff see internal order financials?	Controls order costs/margin scope.	Restrict to owner/admin/accounting and explicitly granted production managers.
Can webstore managers see payouts?	Defines owner-manager financial boundary.	Default no; owner sees payouts, manager sees orders/catalog unless granted.
Should customers see status timeline details?	Controls portal clarity versus internal confidentiality.	Show customer-safe milestones only, not internal production notes or internal assignee details.

Phase 0 Master Foundation

Part 3: Module Boundaries, API Naming & Design System

Detailed implementation instructions for Steps 8-10

Purpose

This document locks the remaining Phase 0 rules before module rebuilding starts: where responsibilities live, how frontend and backend communicate, and which layout/design primitives every module must reuse. These rules keep SignGuy AI from becoming a collection of attractive but mutually suspicious mini-apps.

Document Control	Value
Scope	Phase 0, Steps 8-10 only
Applies To	Every current module, future module, portal, add-on, API endpoint, component, route, and shared service
Predecessors	Part 1: folder structure, routes, tenant ownership, terminology cleanup; Part 2: event/activity logs, settings resolution, permissions
Architecture Source	APPSTRUCTURE MD architecture audit: recommended module-oriented structure, dependency map, route cleanup findings, and final navigation structure
Primary Principle	A module owns its workflow and domain rules; shared systems own shared behavior; no module bypasses these contracts.
Out of Scope	Building feature-specific screens, final reports, individual add-on workflows, production UI, public marketing design, or visual brand artwork

How This Document Fits Phase 0

Steps 8-10 turn the earlier foundation decisions into rules builders can actually follow. Folder structure says where code goes. Route structure says where it is reachable. Tenant, event, settings, and permission rules say how it behaves. This document says which module owns a decision, which API contract exposes it, and which design parts all screens must share.

Step	Decision	Mandatory Result
8	Module boundaries	Every feature has one home, one owner, and a defined contract with other modules.
9	API naming rules	Every endpoint uses consistent resource names, tenant-safe context, validation, errors, and versioning discipline.
10	Design system and layout primitives	Every screen uses a shared shell, shared components, predictable action patterns, and accessible states.

Phase 0 Exit Rule

Do not begin a major module rebuild until the team can answer: Who owns this rule? Which API contract serves it? Which shared primitive renders it? When those answers are unclear, the next feature is not ready to build.

Step 8 - Module Boundaries

Objective: define exactly what each major module owns, what it may read or request from another module, what it must never edit directly, and which shared systems it must use. The architecture audit found overlapping routes, duplicate top-level areas, legacy pages, and data that had drifted into the wrong home. This step prevents those issues from returning in cleaner fonts.

8.1 The Boundary Rules

- Every business capability has one primary owning module. Ownership means the module defines the record lifecycle, validation, domain rules, write service, and authoritative API.
- A module may display related data from another module, but it does not silently become the source of truth for that data.
- Cross-module changes go through a published service/API or a domain event. No direct database writes into another module's tables/collections.
- Shared systems own shared behavior. Modules must not invent private versions of permissions, activity history, settings merging, notifications, uploads, search, or tenant resolution.
- UI composition is allowed. Data ownership theft is not. An Order Detail page may render invoice, proof, document, and task summaries without owning invoices, proofs, documents, or tasks.
- When a workflow crosses modules, one module is the workflow coordinator only for that workflow. It does not absorb the other modules' records or rules.

Ownership Test

Ask: "If this record is wrong, which module's service owns correcting it?" If the answer is "several pages" or "whatever screen edited it last," the boundary is broken.

8.2 Types of Ownership

Ownership Type	Meaning	Example
Domain ownership	Owns the lifecycle, validation, writes, statuses, and authoritative API for a record.	Orders owns Order; Invoices owns Invoice; DocuLink owns Document record.
Workflow coordination	Coordinates a cross-domain process but does not take over each participant record.	Order conversion coordinates Customer + Quote + Order + Order Items + pricing snapshot.
Read-model composition	Combines summaries from several modules for one screen.	Customer Detail shows orders, invoices, proofs, messages, documents, and appointments.
Shared platform ownership	Owns reusable cross-cutting behavior.	Permissions, activity events, resolved settings, uploads, notifications, search.
Integration ownership	Owns provider-specific configuration, webhook handling, and normalized provider state.	Stripe Connect integration owns external account state; Billing consumes its results.
Presentation ownership	Owns UI layout but not data authority.	Dashboard cards, portal summary views, list filters, table layouts.

8.3 Canonical Module Ownership Map

Module / Domain	Owns	Does Not Own	Required Shared Systems
Identity & Access	Auth sessions, login, account recovery, user identities, organization membership, roles/permissions assignments	Customer profile details, employee payroll, module business records	Tenant context, permissions, audit events
Settings & Entitlements	Resolved settings read model, company configuration, feature entitlements, user preferences boundary	Pricing calculations, provider secrets, workflow-specific configuration	Settings resolver, permissions, audit events
Customers / CRM	Customer profile, contacts, customer relationship status, lead conversion, customer portal linkage	Orders, invoices, documents, messages, webstore catalog	Files, events, notifications, portal access
Quotes	Quote lifecycle, quote version/status, quote line items, quote acceptance/conversion request	Canonical order, invoice, production work	Pricing, customers, documents, events
Orders	Order lifecycle, order status, customer linkage, order-level data and coordination	Invoice lifecycle, document lifecycle, task lifecycle, payment provider config	Pricing snapshots, events, files, notifications
Order Items & Work Orders	Order Item lifecycle, product/service execution detail,	Inventory ledger, employee time entries, invoices	Orders, production, documents, pricing

Module / Domain	Owns	Does Not Own	Required Shared Systems
	Work Order Summary generation		
Production	Production board, production tasks/stages, assignments, work readiness	Canonical order pricing, proof approval record, employee payroll	Orders, order items, tasks, calendar, events
Proofs & Approvals	Artwork proof records, approval/revision lifecycle, approval evidence	Artwork file storage implementation, orders, documents generally	Files, documents, customer portal, events, notifications
Invoices & Payments	Invoice lifecycle, payment state, customer-facing invoice records, payment reconciliation	Stripe provider credentials, subscription entitlement billing	Orders, customers, integrations, events
DocuLink	Document records, document templates, generated document metadata, portal visibility rules	Raw object storage provider, customer/order business fields	Files, forms, signatures, notifications, events
Forms / Questionnaires / Signatures	Form templates, response records, signature requests and evidence	Customer master record, document storage provider	Customers, DocuLink, notifications, events
Inventory & Purchasing	Inventory items, locations, vendors, POs, receiving, material movement	Order ownership, product pricing authority	Orders/order items, pricing, events
Productivity	Tasks, Kanban state, appointments, internal work coordination	Production-stage rules, employee payroll	Orders, customers, production, events, notifications
Team / Payroll / Time	Employees, time entries, payroll work records, employee schedules	Identity account core, task lifecycle, order lifecycle	Identity, permissions, events, employee portal
AI Platform	Prompt library, credit ledger, AI run records, assistant orchestration	Customer/order source records, pricing authority	Permissions, events, files, settings, entitlements
Webstores	Store lifecycle, store configuration, per-store catalog, public storefront, store owner workflow	Canonical internal order schema, Stripe provider security implementation	Orders, customers, billing/payments, files, forms, events
Wrap Lab	Vehicle-wrap-specific Order Item extension, inspections, vehicle details, wrap packet workflow	Canonical customer/order/payment/doc systems	Order items, pricing, production, docs, proofs, files
Portals	Role-specific presentation and controlled actions for customers/employees/owners	Underlying domain records and business rules	Permissions, portal policies, all consuming domains
Platform Admin	Tenant oversight, platform analytics, platform-level audit, global settings	Tenant business operations data ownership	Identity, entitlements, audit events, analytics

8.4 Cross-Module Interaction Rules

Situation	Correct Pattern	Forbidden Pattern
Order needs customer name/address	Orders reads Customer summary through CRM service/read model and stores an approved snapshot only where needed.	Orders edits customer profile fields directly in its own repository.
Invoice becomes paid	Payments/Invoices updates invoice state, emits invoice.payment_received; Orders consumes event to update an order summary badge.	Orders marks invoice paid because its page got a Stripe callback.
Proof is approved	Proofs owns approval state and emits proof.approved; Orders/Production react through an event or query.	Production changes approval record directly to move its board card.
Document is attached to order	DocuLink creates document attachment link; Order Detail reads document summary.	Orders creates a second document table and bypasses DocuLink.
AI drafts a quote email	AI creates an AI run/result; Messaging sends after user approval; Quote/Order may link context.	AI module writes directly into email provider and logs nowhere.
Webstore checkout completes	Webstores owns checkout translation and requests creation of canonical Order(s) through Orders service.	Webstore creates a parallel “store order” that never maps to canonical orders.
Wrap inspection is completed	Wrap Lab updates its extension record and emits event; Order Item summary reflects it.	Wrap Lab overwrites generic Order Item fields with vehicle-only data.
Dashboard needs metrics	Dashboard queries module read models/analytics service.	Dashboard reaches into every module database table and calculates whatever it feels like.

8.5 The Module Contract

Every module documented and rebuilt after Phase 0 must publish a short Module Contract. This becomes the first section of that module’s rebuild documentation.

Contract Section	Required Content
Purpose	What business outcome the module owns.

Contract Section	Required Content
Owned entities	Primary records, statuses, validations, and lifecycle rules.
Public operations	Actions other modules may request, such as create order from quote or attach document to order.
Read models	Safe summaries other modules may read without learning private schema details.
Published events	Events emitted after successful committed changes.
Consumed events	Events it reacts to and what reaction occurs.
Dependencies	Required upstream modules and shared systems.
Permissions	Capabilities required for internal, portal, and admin actions.
Settings	Which settings domains it reads and which it owns.
Prohibited access	Tables/services it must not write directly.
Migration notes	Legacy routes, fields, aliases, redirects, and data mapping.

8.6 Boundary Rules for Common Trouble Spots

- Orders vs Order Items: Orders owns the commercial container and overall lifecycle. Order Items own item-specific execution details. “Work Order Summary” is generated from Order Items and production context; no new code may reintroduce Job Ticket as the canonical entity.
- Pricing vs Inventory: Pricing owns how material/labor data becomes a quote or estimated selling price. Inventory owns what material exists, what it costs, where it lives, and how stock changes. Pricing may read current inventory cost; it does not mutate stock.
- DocuLink vs Files: DocuLink owns document metadata, categories, template relationships, attachment links, portal visibility, and lifecycle. File Storage owns physical upload/download/object references. No module stores random file metadata in a private field.
- Customer Portal vs Customer CRM: CRM owns the customer record. Customer Portal owns authenticated customer presentation and limited customer actions. It cannot establish its own alternate customer profile source.
- Webstores vs Orders: Webstores owns storefront configuration and checkout translation. Orders remains canonical for shop operations and production. This is mandatory so store orders do not become invisible to the rest of the app.
- AI vs Every Other Module: AI may read explicitly authorized context and produce suggestions or structured proposals. The receiving domain validates and commits any actual change. AI does not bypass permissions or write arbitrary records.
- Platform Admin vs Tenant Admin: Platform Admin governs platform-wide services and tenant oversight. Tenant Admin governs its own organization. Tenant Admin must never reach platform-level records; Platform Admin access must be explicit and audited.

8.7 Boundary Enforcement Checklist

- Create a module contract for every module before implementing its new backend service.
- Place module-owned models, services, routes/controllers, components, and tests in that module’s folder according to Phase 0 Part 1.
- Ban cross-module direct model imports for writes. Cross-module reads must use a published read service/query adapter where practical.
- Add architecture tests or lint rules that flag imports from another module’s internal/private folders.
- Require event publication only after the source transaction commits successfully.
- Require every cross-module workflow to name its coordinator and failure/retry behavior.
- Document every legacy exception; do not silently reproduce it.

Step 9 - API Naming Rules and Service Contracts

Objective: establish an API language that stays understandable as SignGuy AI grows into customer portals, employee portals, Webstores, Wrap Lab, AI services, and platform administration. The current audit identified legacy aliases, mixed “job ticket” terminology, ambiguous prefixes, and routes that do not match component intent. This step makes the HTTP surface deliberate rather than archaeological.

9.1 API Principles

- Use resource-oriented, plural nouns for collections and stable IDs for individual records.
- Use canonical business terms in every new endpoint: orders, order-items, work-orders, quotes, invoices, customers, proofs, documents. Do not create new job-ticket endpoints.
- Tenant context comes from authentication/session middleware. Do not accept a tenant_id in ordinary tenant-facing URL paths or request bodies.
- Endpoints must have one clear owner module. A controller is not a dumping ground for convenience actions from unrelated modules.
- Validate input at the API boundary, authorize before action, execute domain service, commit transaction, publish event, return stable response. In that order.
- Use consistent pagination, filtering, sorting, error shapes, idempotency, and request/correlation IDs across all modules.
- Public and portal endpoints must live in separate access-zone namespaces from internal tenant APIs. A public storefront is not an internal API with a missing lock.

9.2 Namespace and Access-Zone Structure

/api/v1/auth/...	authentication and session actions
/api/v1/me/...	authenticated current-user preferences/profile
/api/v1/internal/...	tenant-authenticated staff/admin operations
/api/v1/customer-portal/...	authenticated customer portal operations
/api/v1/employee-portal/...	authenticated employee portal operations
/api/v1/owner-portal/...	authenticated webstore owner/manager operations
/api/v1/public/...	public forms, public storefront reads, public checkout handoff
/api/v1/platform-admin/...	platform staff operations only
/api/v1/integrations/...	secure provider-specific callbacks/admin operations
/api/v1/system/...	tightly controlled health/status/internal diagnostics

Do Not Mix Zones

The same resource may appear in multiple zones, but the endpoint contract must be tailored to that zone. Example: internal orders may include cost and margin; customer-portal orders may include only customer-safe order status and documents.

9.3 Canonical Resource Names

Concept	Canonical Endpoint Noun	Legacy / Avoid
Customer	customers	clients if it means the same record
Quote	quotes	estimates unless formally a distinct concept
Order	orders	jobs when referencing commercial order
Order Item	order-items or nested /orders/{order_id}/items	job-tickets
Work Order Summary	work-orders or /order-items/{id}/work-order	job-ticket detail
Artwork Proof	proofs	artwork-approval-file
Invoice	invoices	billing record when invoice is meant
Document	documents	files when document metadata/lifecycle is meant
Questionnaire	questionnaires	forms if it has specific questionnaire workflow
Appointment	appointments	schedule-entry when appointment is meant
Employee	employees	users when employee work record is meant
Webstore	webstores	stores when the system also has physical store concept
Webstore Product	products nested under webstore	global catalog item unless truly global
Vehicle Wrap Extension	wrap-projects or /order-items/{id}/wrap	wrap command center record as a generic order
AI Run	ai-runs	assistant message when it is an execution record

9.4 URL Shape Rules

Rule	Good	Avoid
Use plural collections	GET /api/v1/internal/orders	GET /api/v1/internal/order
Use IDs, not display names	GET /orders/ord_123	GET /orders/John-Smith-sign
Nest only for real containment/context	GET /orders/ord_123/items	GET /customers/cus_1/orders/ord_1/invoices/inv_1/proofs
Use query strings for filter/sort/pagination	GET /orders?status=in_production&sort=-created_at&limit=50	GET /orders/in-production/page/2/descending
Use named actions only when no CRUD noun fits	POST /quotes/q_1/convert-to-order	POST /quotes/q_1/doConversion
Keep legacy aliases isolated	GET /legacy/job-tickets/{id} -> redirect/adaptor only	New UI calling /jobtickets forever
Keep portal zones explicit	GET /customer-portal/orders/{id}	GET /orders/{id}?portal=true

9.5 CRUD and Action Semantics

Intent	Method / Pattern	Notes
List records	GET /resource	Filter/search/paginate through query params.
Read one record	GET /resource/{id}	Return one stable public DTO for that access zone.
Create	POST /resource	Server creates ID, timestamps, owner context, and events.
Replace rarely	PUT /resource/{id}	Use only if full replacement semantics are real.
Partial update	PATCH /resource/{id}	Preferred for field/status changes; validate allowed fields.
Archive / soft delete	POST /resource/{id}/archive or PATCH status	Prefer archive where business/audit history matters.
Hard delete	DELETE /resource/{id}	Only for safe data classes and authorized roles.
State transition	POST /resource/{id}/actions/{action}	Use when the action has validation/workflow effects beyond a field edit.
Bulk operation	POST /resource/actions/bulk-{action}	Return per-item results and idempotency behavior.
Export	POST /resource/exports or GET /resource/export	Use async job for large exports; never block forever.

9.6 Mandatory API Request Lifecycle

1. Resolve actor, access zone, tenant/platform context, and request/correlation ID from trusted middleware.
2. Authorize the requested capability through the shared permissions system.
3. Validate request body/query/path values with typed schema.
4. Call the owning module service. Controllers do not contain major business logic.
5. Run database transaction where multiple writes must succeed together.
6. Commit the transaction.
7. Publish domain event(s) through the shared event writer after commit.
8. Return a stable response DTO with server-calculated values, revision/version information where useful, and a correlation ID.

9.7 Response Shape

```
{
  "data": { "id": "ord_123", "status": "in_production", "...": "..." },
  "meta": {
    "request_id": "req_...",
    "revision": "2026-07-02T12:10:00Z",
    "permissions": { "can_edit": true, "can_archive": false }
  }
}

{
  "data": [ ... ],
  "meta": { "page": 1, "limit": 50, "total": 248, "next_cursor": "..." }
}
```

The exact DTO fields vary by module and access zone. The envelope and error conventions do not. Avoid returning raw database models simply because it is faster for the first afternoon.

9.8 Error Contract

HTTP Status	When to Use	Required Error Code Examples
400	Malformed request that cannot be interpreted	invalid_request
401	No valid authentication/session	authentication_required
403	Authenticated but lacks permission or record access	permission_denied, tenant_access_denied
404	Resource does not exist or must be hidden from caller	resource_not_found
409	Conflict with current state/revision/idempotency	status_conflict, duplicate_request, revision_conflict
422	Input is structurally valid but violates domain rule	validation_failed, cannot_convert_quote, invalid_status_transition
429	Rate/credit/request limit reached	rate_limited, ai_credit_limit_reached
500	Unexpected server fault	internal_error
502/503	Provider or temporary dependency problem	provider_unavailable, service_temporarily_unavailable

```
{
  "error": {
    "code": "invalid_status_transition",
    "message": "An order cannot move to production until required proof approval is complete.",
    "fields": { "status": "proof_approval_required" },
    "request_id": "req_..."
  }
}
```

9.9 Pagination, Filtering, Search, and Sorting

- Use cursor pagination for large/changing collections where stable ordering matters; page/limit is acceptable for smaller administrative lists if consistently implemented.
- Use one convention for sorting: sort=created_at or sort=-created_at for descending.
- Use named filters: status=, customer_id=, assigned_to=, from=, to=, include_archived=.
- Use q= for simple text search and a module-specific advanced filter object only when truly necessary.
- Return total only where it is affordable and meaningful. For expensive large datasets, return next_cursor and estimated/omitted totals deliberately.
- Never make UI components invent their own filter semantics around raw endpoint quirks.

9.10 Idempotency, Concurrency, and Retries

Case	Required Rule
Payment/checkout/webhook creation	Require provider event IDs and idempotency keys; duplicate delivery must be safe.
Create actions from UI	Support Idempotency-Key for actions likely to be retried or double-clicked, especially invoice send, document send, quote conversion, checkout handoff.
Edits to important records	Use updated_at/revision or optimistic concurrency token for forms where overwrite risk is meaningful.
State transitions	Reject stale/invalid transition with 409/422 and return current state.
Async jobs	Store job/run ID, status, error, retry count, and correlation ID.
Events	Event consumers must be idempotent; receiving the same event twice cannot create duplicate invoices/tasks/messages.

9.11 API Versioning and Deprecation

- Start new external-facing contracts under /api/v1. Internal implementation can evolve behind the service layer without exposing raw changes.
- Do not version an endpoint for every field addition. Version only when behavior or response meaning becomes incompatible.

- Legacy endpoints remain read-only adapters or redirects during migration. Label them clearly, log usage, and set a removal target.
- Every deprecated endpoint needs: replacement endpoint, migration note, caller inventory, monitoring, warning log, and removal condition.
- No new frontend code may call legacy endpoints. Add lint/test coverage if necessary.

9.12 API Documentation and Test Requirements

Deliverable	Required Content
API registry	Endpoint, owner module, access zone, request schema, response DTO, permissions, events emitted, deprecation status.
OpenAPI or equivalent	Machine-readable contract for all active endpoints.
Contract tests	Validate response and error shapes across modules.
Authorization tests	Prove tenant/portal/platform users cannot cross access boundaries.
Tenant isolation tests	Prove a valid ID from another tenant is not readable or writable.
Idempotency tests	Prove duplicate submits/webhooks do not duplicate business records.
Migration tests	Prove legacy aliases map correctly or fail safely during retirement.

Step 10 - Design System and Layout Primitives

Objective: define the reusable visual and interaction foundation for the rebuilt application. The architecture audit found navigation overlap, pages that belong in the wrong area, overloaded tabs, and inconsistent layouts. This step creates a reusable shell and a small set of standard screen patterns so every module does not invent a new dashboard, form, card, table, empty state, ribbon, or portal navigation.

Scope Guard

This is not a final brand board or a request to decorate every page now. It defines reusable structure, spacing, behavior, components, and accessibility. The brand colors, logo treatment, and module-specific visuals can evolve without changing the system underneath.

10.1 Design System Principles

- Use one app shell with consistent top navigation, page header, contextual ribbon/actions, content area, feedback layer, and responsive behavior.
- Prefer recognizable business-software patterns: dashboard, list, detail, create/edit form, board, calendar, timeline, report, portal, and settings. Do not invent a new navigation metaphor for every department.
- Keep dense business information readable. Compact does not mean cramped; spacious does not mean acres of empty white nothing.
- Show the most important next action and record status without forcing users through five tabs to discover what needs attention.
- Use progressive disclosure: summary first, details when opened. Keep previously entered information visible in compact read-only form rather than repeating giant forms.
- Build for desktop-first shop operations while remaining usable on tablet and mobile. Portals require stronger mobile support than internal production administration.
- Accessibility and permission state are part of the component design, not last-minute stickers.

10.2 Application Shell

Shell Layer	Responsibility	Rules
Global top navigation	Switch between major application areas	Home icon returns to main dashboard. Keep logical areas stable; do not duplicate module links across top nav and ribbon.
Contextual ribbon / action bar	Fast access to frequent actions relevant to current app area	Office-style compact action bar; maximum 12 items per tab/group; icon + readable label; no truncated mystery labels.
Area sub-navigation	Navigate within a major area	Use tabs/sidebar only when it clarifies a bounded module set. Do not create a third nav layer for every page.
Page header	Title, breadcrumb, record status, key actions	Header should answer: Where am I? What record is this? What state is it in? What can I do next?
Content canvas	Render module screen pattern	Use shared spacing, grid, cards, tables, forms, feedback states.
Feedback layer	Toasts, inline validation, banners, modals, confirmations	Shared primitives only; no ad hoc browser alerts for business workflow.
Portal shell	Customer/employee/owner-safe navigation and actions	Separate shell/navigation from internal app. Only portal-approved features and fields appear.

10.3 Standard Page Patterns

Pattern	Use For	Required Elements
Dashboard	Area overview and actionable summaries	KPI cards, Action Required, quick actions, limited relevant lists, clear empty state.
List	Customers, orders, invoices, documents, inventory, employees	Search, filters, column controls, sort, pagination, row actions, bulk actions only when safe.
Detail	Customer, order, invoice, webstore, employee, wrap project	Identity header, status, summary, related records, timeline, primary actions, contextual tabs only when needed.
Create / Edit Form	New records and controlled edits	Sectioned form, required labels, inline validation, save/cancel, unsaved-changes guard,

Pattern	Use For	Required Elements
		progressive disclosure.
Board	Production, Kanban, task workflow	Clearly defined columns/states, drag actions with permission checks, filters, accessible non-drag fallback.
Calendar	Appointments, employee schedules, install scheduling	Day/week/month views as appropriate, filters, click-to-view, no hidden scheduling logic in UI.
Timeline	Activity/audit history	Chronological events, actor, timestamp, action, safe detail, filters.
Report	Financial/operational metrics	Date range, filters, definitions, export, data freshness, empty state explaining no records.
Settings	Domain configuration	Grouped settings, owner/permission explanation, validation, save feedback, audit-aware changes.
Portal	Customer/employee/owner self-service	Clear next steps, simple navigation, mobile-first actions, no internal-only data.

10.4 Shared Component Inventory

Component	Required Behavior	Used By
AppShell	Global nav, ribbon mount, content bounds, responsive layout	Internal app
PortalShell	Portal-specific nav, identity, secure logout, mobile layout	Customer/employee/owner portals
PageHeader	Title, breadcrumb, status chip, primary/secondary actions	All detail/list/settings pages
ActionRibbon	Grouped actions, max 12 visible actions, overflow behavior, keyboard access	Major app areas/modules
StatusBadge	Consistent status color/icon/text mapping	Orders, proofs, invoices, tasks, webstores, payroll
DataTable	Columns, sort, filters, search, pagination, saved views where appropriate	Lists and reports
FilterBar	Common filter chips, reset, count, shareable URL state	Lists/boards/reports
RecordSummary	Compact read-only field group with inline edit where allowed	Details, review screens, cross-module summaries
FormSection	Section title, help text, fields, validation summary	Create/edit/settings forms
Field primitives	Text, select, combobox, date, money, phone, address, attachment, assignee, status	All forms
EmptyState	Explains why empty + permitted action to fix it	All data views
LoadingState	Skeleton/spinner with readable context	All async data screens
ErrorState	Plain-language error, retry, support/reference ID if relevant	All async data screens
ConfirmDialog	High-risk confirmation with clear irreversible language	Deletes, sends, status transitions, payments
ActivityTimeline	Shared event rendering and filter controls	Details, admin audit, portals where allowed
AttachmentPicker	Upload/select/link file via shared file service	Orders, documents, proofs, forms, wraps
MoneyDisplay	Currency, rounding, tax/margin visibility rules	Pricing, quotes, orders, invoices, reports
PermissionGate	Consistent hide/disable/explain behavior based on capabilities	All internal/portal UI

10.5 Layout Grid, Density, and Spacing

- Use a consistent desktop content max width and responsive gutters. Avoid layouts that become unreadably wide on large monitors or cramped on normal shop laptops.
- Use a small spacing scale consistently, such as 4, 8, 12, 16, 24, 32 pixels. Components should not invent random gaps.
- Use a reusable 12-column desktop grid where complex layouts need it; standard pages may use one-column, two-column, or three-column patterns.
- Use three-column dashboards only where the content remains scannable. Do not make a single widget span the entire page unless it truly needs to, such as a calendar or production board.
- Use compact summary cards, thin visual distinction, and clear headings. Large empty cards with one sentence waste vertical space and patience.
- Forms should use readable widths: short fields grouped together, addresses and descriptions full width, labels above fields, help text where needed.

- On narrow screens, collapse multi-column forms to one column and move less critical supporting panels below primary content.

10.6 Navigation and Action Rules

Rule	Implementation
Home	A home icon returns to the main dashboard. Do not make multiple competing “dashboard” top-level destinations.
Major navigation	Use the approved major areas; do not recreate Finance, Reports, Inventory, or Community as duplicate top-level destinations when they belong under an area.
Ribbon	Keep ribbon actions to common actions and shortcuts, not full site navigation. Use icons + labels. Labels must not truncate.
Detail tabs	Use tabs only for meaningful record sections, usually 3-7. Example: Order Detail may have Overview, Items, Proofs, Production, Documents, Messages, Finance, Activity if all are justified.
Back behavior	Use breadcrumbs and consistent back-to-list behavior. Do not rely on browser history for essential workflow navigation.
Primary action	One clear primary action per screen state. Example: Send Quote, Create Order, Request Approval, Mark Paid.
Secondary actions	Place less common actions in contextual menus; avoid action clusters that make every button look equally urgent.
Destructive actions	Use explicit confirmation and explain related effects. Archive where audit/history matters.

10.7 Form and Data Entry Rules

- Enter information once and reuse it. Customer details, company details, item details, and order references should populate where already known.
- When a user changes tabs or steps, keep completed information visible as compact summary blocks. Do not force them to re-open giant labeled input containers merely to confirm what they entered.
- Autosave may be used only where revision/conflict behavior is clear. Critical financial, pricing, and state-transition actions need explicit save/confirm behavior.
- Every required field must have a visible required indicator and a useful validation message near the field plus an optional form-level summary.
- Use searchable comboboxes for large reference lists such as customers, materials, products, and employees. Support creating a new related record only when it does not break the workflow.
- Use money/date/phone/address controls that normalize formats while preserving user understanding.
- Prevent accidental duplicate submissions with disabled submit state, idempotency key, and visible progress feedback.

10.8 Status, Color, and Feedback System

Element	Rule
Status badges	Use text plus color/icon. Never communicate state with color alone.
Semantic colors	Reserve semantic colors for meaning: success, warning, error, info, neutral. Module branding may not override financial/payment risk colors.
State mapping	Define centralized status-to-label-to-color maps per domain. Do not let every component choose random colors for “approved.”
Toasts	Use for short confirmation or recoverable notices. Do not hide critical failures in a toast that disappears before anyone reads it.
Inline alerts	Use for blocked state, missing setup, payment issue, permission denial, or workflow requirement.
Banners	Use sparingly for account-wide issues, maintenance, onboarding, or important action-required notices.
Loading	Show skeletons or concise progress indicators. Avoid blank white panels that look broken.
Empty	Explain what is absent, why it may be absent, and the permitted next action.

10.9 Accessibility and Responsive Requirements

- All interactive controls must be keyboard reachable, have visible focus, and have an accessible label.
- Color contrast must meet readable standards for text, status badges, buttons, chart labels, and disabled/hover/focus states.

- All icons used as actions must have accessible names; icon-only actions need tooltips plus labels for screen readers.
- Errors must be announced and tied to the relevant field. Do not rely on red borders alone.
- Tables need responsive fallback: horizontal scroll with sticky key column, card transformation only when it keeps comparison usable, or a focused mobile detail view.
- Boards need a non-drag alternative for moving work, such as action menu/state selector.
- Customer, employee, and owner portals must be tested on common mobile widths as a release gate.
- Do not hide critical data on mobile just to make the page look less busy. Prioritize and collapse, but keep access available.

10.10 Module Design Handoff Requirements

Before a Module UI Starts	Required Decision
Page map	Which standard patterns are needed: list/detail/form/board/calendar/report/portal.
Actions	Primary, secondary, destructive, bulk, and permission-limited actions.
Data contract	Which API DTO powers each view and which loading/error/empty states exist.
Shared components	Which primitives are reused and which truly new component is justified.
Responsive behavior	What changes at tablet/mobile widths.
Accessibility	Keyboard path, focus order, labels, error behavior, non-color cues.
States	Loading, empty, partial setup, permission denied, error, archived, disabled, offline/provider failure where relevant.
Visual tests	Screenshots/story examples for key states and regression coverage.

10.11 Design System Governance

- Create a small shared component library and document it before modules start duplicating buttons, tables, forms, badges, and modal behavior.
- Every proposed new primitive must answer: Why cannot an existing primitive be configured? Which three or more modules will use it? If only one, it may belong inside that module.
- Changes to global navigation, density, form behavior, status semantics, or permission presentation require review because they affect the entire product.
- Module-specific styling lives in module scope; global tokens and primitives live in shared design-system folders.
- Add visual regression checks for the app shell, list/detail/form patterns, and portal shells. This saves you from discovering that a small “fix” moved every header six inches down the page.

Phase 0 Part 3 Completion Gates

Steps 8-10 are complete only when these conditions are true. A document nobody follows is merely a very organized form of wishful thinking.

- Every major module in the master module inventory has an initial Module Contract naming its owned entities, public operations, dependencies, events, and prohibited direct access.
- The canonical ownership map is approved, with explicit decisions for known overlap areas: Orders/Order Items, Pricing/Inventory, DocuLink/Files, Portals/Source Domains, Webstores/Orders, Wrap Lab/Order Items, AI/Domain Writes, and Platform Admin/Tenant Admin.
- New API namespaces, canonical nouns, error envelope, response envelope, pagination/filter/sort conventions, and ID conventions are implemented as shared scaffolding.
- At least one internal endpoint, one portal endpoint, one public endpoint, and one platform-admin endpoint have contract tests demonstrating the access-zone rules.
- Legacy endpoint inventory exists with replacement mapping and deprecation status. New code is blocked from adding job-ticket terminology or legacy endpoint calls.
- AppShell, PortalShell, PageHeader, ActionRibbon, DataTable, FormSection, status badges, feedback states, PermissionGate, and ActivityTimeline primitives are implemented or formally stubbed with interfaces.
- One sample list page, detail page, form page, board/calendar page, and portal page use the shared primitives and pass visual/accessibility review.
- Every Phase 0 document is linked from the rebuild root README/architecture index and is treated as mandatory implementation guidance.

Emergent Implementation Handoff Prompt

Use This Prompt

Send this to Emergent only after providing Phase 0 Parts 1, 2, and 3. It is intentionally restrictive. The point is to establish the substrate before another module starts making “temporary” exceptions that survive for two years.

You are implementing Phase 0, Part 3 of the SignGuy AI rebuild foundation.

Use the attached Phase 0 Part 1, Part 2, and Part 3 documents as mandatory architecture rules. Do not begin rebuilding feature modules yet.

Implement only the foundations in Part 3:

1. Module boundary enforcement and Module Contract template/scaffolding.
2. API namespace, endpoint naming, response/error envelope, validation, pagination/filter/sort, idempotency, and deprecation scaffolding.
3. Shared design system primitives and application/portal shell foundations.

Required outcomes:

- Create a module contract registry/template for every major module.
- Establish canonical resource naming with Order, Order Item, and Work Order terminology. Do not create new Job Ticket names.
- Implement access-zone separation for internal, customer portal, employee portal, owner portal, public, platform admin, integrations, and auth APIs.
- Implement one standard response envelope and one standard error envelope.
- Add typed request/response validation and tenant/permission middleware hooks to all new endpoints.
- Add a legacy endpoint inventory with adapters only where needed; do not let new frontend code call legacy paths.
- Create shared UI primitives: AppShell, PortalShell, PageHeader, ActionRibbon, DataTable, FilterBar, RecordSummary, FormSection, Empty/Loading/Error states, ConfirmDialog, StatusBadge, PermissionGate, and ActivityTimeline.
- Build one internal sample list/detail/form page and one portal sample page using these primitives only to prove the foundation.

Constraints:

- Do not move ownership of records across modules without documenting it in the module contract.

- Do not directly write another module's tables/models from a feature module.
- Do not add alternate navigation patterns, settings merge logic, role checks, event schemas, or API error formats.
- Do not use raw database models as API responses.
- Do not rebuild Webstores, Wrap Lab, payroll, AI tools, reports, or public marketing pages in this task.

Return:

1. Files created/changed with exact paths and what each does.
2. Module boundary/contract registry.
3. API convention reference with example endpoints.
4. Shared component inventory and sample pages.
5. Tests run and results.
6. Any existing code that conflicts with Phase 0 rules, marked as keep, migrate, deprecate, or remove.
7. A concise "Phase 0 Part 3 Complete / Not Complete" checklist.

Reference Note

This specification is derived from the SignGuy AI architecture audit, particularly its recommended module-oriented folder structure, module dependency map, data/entity ownership findings, route inventory, duplicate/legacy cleanup findings, navigation problems, and recommended final navigation structure. It should be read together with Phase 0 Parts 1 and 2.